Jayesh Ranjan Kesari

AI Engineer · Agentic Systems & LLM Infrastructure

jrkesari@gmail.com · github.com/Jrkesari · linkedin · jayesh.cv

PROFILE

AI engineer building production agentic systems since 2024. I work on the LLM platform behind a multi-tenant enterprise product: model routing across providers, an MCP tool server, skills people write and share, RAG over documents and spreadsheets, and token-level cost tracking.

SKILLS

Languages	Python, SQL, TypeScript, Go
Agentic	MCP (client & server), tool-calling loops, agent skills (Claude Code & Codex), evals, LangGraph, LangChain, CrewAI
LLM	Azure OpenAI, Anthropic Claude, Google Gemini, NVIDIA NIM, self-hosted models on vLLM; prompt caching, structured outputs, RAG, vision
Backend	FastAPI, asyncio, Pydantic, gRPC, REST
Data	text-to-SQL, DuckDB, SQLite + sqlite-vec, ClickHouse, Redpanda, FAISS, pandas
Platform	Docker, Azure, Microsoft Graph, OpenTelemetry, GitHub Actions, pytest, uv

EXPERIENCE

Netision Technology LLP · AI Engineer

May 2025 – present · Noida · on-site

Agent runtime and tools

- Wrote the agent loop that runs every AI query: tool schemas, capped iterations, result truncation, and a last tool-free call so the model has to answer in text.
- Set up a tag-based tool contract where tools declare what they do and what they need, so adding one takes no runtime changes.
- Shipped skills in the product: people write and share their own, and the model loads one when a task calls for it. A skill shapes the answer, never the tools or where data goes.
- Removed an extra billed round on capped turns by offering file tools only when the prompt actually asks for a file.
- Added cooperative cancellation across the pipelines after finding that stopped queries were still writing their results.

Multi-provider model layer

- Built a shared library and model registry: 17 models across 6 providers, picked per request instead of baked into the pipeline; 101 releases in four months.
- Got prompt caching working with ordered breakpoints, bringing repeat tool-loop rounds down to a fraction of the first round's cost.
- Tracked down a content-filter block that only showed up through the proxy, then confirmed the fix on live calls: blocked calls went from 3/12 to 0/12.
- Self-hosted a 34B open model on one GPU with vLLM to answer "should we?" with numbers: tool use went from 1/8 to 8/8 after rewriting its chat template, and fell to 39/60 at 35 tools.

Token accounting

- Built a per-user cost ledger that meters every paid call (chat, embeddings, vision, OCR, images) into SQLite with a ClickHouse mirror.
- Made missing usage and unpriced models record as such rather than as zero, so nothing free gets confused with something never measured.
- Read token counts straight from provider payloads so cached-input and reasoning tokens are not lost.

Documents and spreadsheets

- Routed uploads on text density rather than file extension, so image-only decks go to vision transcription and text PDFs to the normal extractor.
- Swapped naive chunking of spreadsheets for SQL over detected tables with schema-card embeddings, and made table discovery progressive: graded accuracy went from 2/8 to 6/8.
- Sandboxed model-written SQL: no external access, read-only statements, scoped to the current conversation's tables.

Connectors, tools and security

- Architected the MCP tool server: 69 tools, including 20 per-user connectors across eight providers, passing delegated tokens as transport headers so the model can never set one.
- Wrote 19 of the connectors (Microsoft 365, Power BI, Google Workspace, Zoho, Databricks, Dynamics 365, Odoo); a teammate wrote Salesforce. The last four haven't met a live account yet.
- Built a deck tool that plans, writes, checks and delivers a native PowerPoint file. Cited sources reach it from code, not from the model, so footnote 3 on a slide is source 3 in the chat.
- Designed host-side approval for outbound mail: Send runs the parked call with a fresh token and no model in the path. A teammate built the first version; I added replies and attachments.

Engineering practice

- Wrote most of the query backend's 67 test modules, covering wire contracts, cancellation, metering and citations.
- Wrote eight project skills for my coding agent: a tiered release gate, a root-cause checklist, and token, dependency and security audits. Merging is never the agent's call.
- Made a known-vulnerable dependency fail the build in three repos, with the advisory scan running daily instead of only on a push.

Earlier at Netision

- Text-to-SQL analytics and root-cause agents over a read-only warehouse; cut a root-cause turn from 77.8 s to 12 s with the report unchanged.
- Backends for an on-prem observability platform in Go and FastAPI: agents, a collector, a probe, and a control plane on Redpanda and ClickHouse.

Coginetics · AI Software Trainee

Dec 2024 – May 2025 · Noida

- Built LLM agents with LangChain, CrewAI and the OpenAI API, where I moved from web development to agents.

Tech Freedom Online · Web Developer Intern

Jul 2024 – Dec 2024 · Remote

- Front-end and back-end work on web applications.

PROJECTS

Published skills, open source. Claude Code and Codex plugins, MIT licensed: a beat-synced video editor and an Android TV fixer that diagnoses before changing anything. `Python, ffmpeg, HyperFrames, ADB`

Intelligent web scraper with MCP. An MCP server that exposes web scraping to any MCP client, probing in parallel to tell static, client-rendered and bot-protected sites apart, then picking a strategy. `Python, Playwright, BeautifulSoup, MCP`

SplitCash: voice-driven finance pipeline. Hindi speech to translated, structured ledger entries; 5 speech-to-text and 3 LLM providers behind one interface. `LangChain, DuckDB, Python`

EDUCATION & CERTIFICATIONS

Bennett University · BCA, Data Science · 9.6/10 — 2022 – 2025

Introduction to Model Context Protocol, Anthropic (2025) · AI Fluency: Framework & Foundations, Anthropic (2025) · Google Data Analytics Professional Certificate · AWS Academy Graduate: Data Engineering, Cloud Architecting, Cloud Foundations